



Projet Machine learning

Pour le projet de Machine learning j'ai choisis de traiter le sujet des transactions bancaires frauduleuses.

1- Compréhension de la problématique

1- a) Présentation de la problématique en explicitant le rôle du ML dans l'étude du thème et en définissant clairement le problème métier et le traduire en tâche ML (régression ou classification).

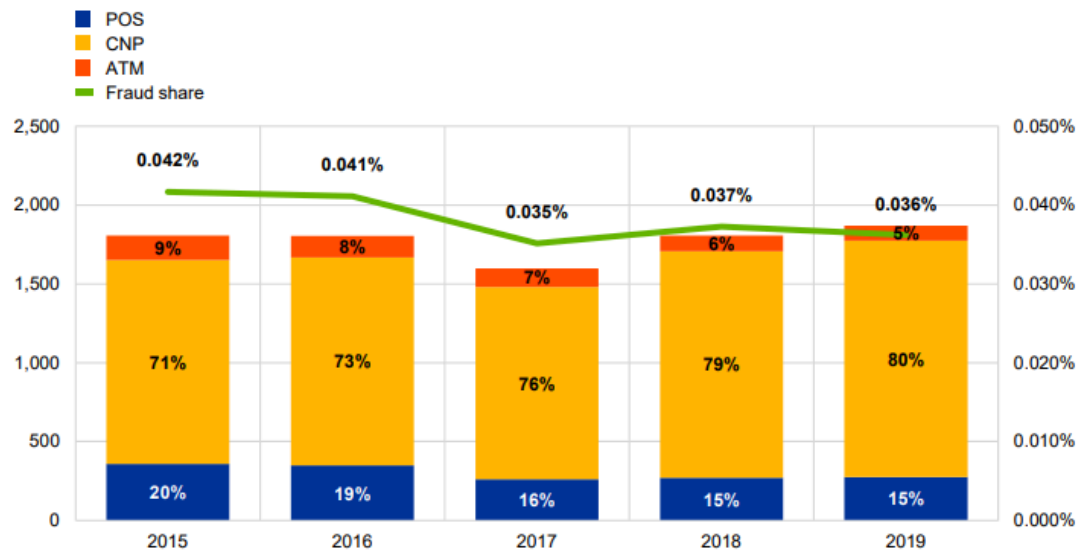
- 1) La fraude des cartes bancaires est la fraude la plus importante parmi les différents cas de fraude qui existe et que malgré le fait que les banques ont implémenté la technologie EMV les fraudeurs se sont tournés vers la fraude CNP (card not present)
 - via les site e-commerce etc...



Chart 1a

Total value of card fraud using cards issued within SEPA

(left-hand scale: total value of fraud (EUR millions); right-hand scale: value of fraud as a share of the value of transactions)



Source: All reporting card payment scheme operators.

Source : https://www.ecb.europa.eu/pub/pdf/cardfraud/ecb_cardfraudreport202110~cac4c418e8.en.pdf

La détection de fraude relative aux cartes de crédits est très importantes pour les institutions financière puisque laisser les client exposé à ce risque sans le prendre en charge peut avoir des conséquences business importantes :

- Pertes financières :
- Dommage réputationnels, perte de productivité, et une expérience client négative qui peut conduire au turnover client
- Des amendes par les régulateurs : par exemple les régulateurs fédéraux américains ont infligé une amende de 255 millions de dollars à Bank of america pour un système de détection de fraude défaillant pendant le covid-19

Source: <https://www.ibm.com/think/topics/fraud-detection>

Rôle du machine learning dans la résolution du problème de détection de fraude : Face à un large volume de transactions, et un nombre très niche de fraude (moins de 2%) où les fraudeurs font tout pour rendre chaque transaction légitime (Vaishnavi Nath Dornadula et al), ce problème métier se traduit par un problème de classification où le modèle devra être



suffisamment entraîné afin de détecter de manière précise les anomalies. Il s'agira précisément d'un problème de classification binaire où la valeur 1 de la target correspond à une fraude et la valeur 0 de la target correspond à une transaction normale. Pour bien évaluer les modèles dans le cadre de cette problématique de classification, les auteurs recommandent l'utilisation du MCC. Le MCC est un coefficient qui permet d'apprécier la qualité d'un modèle de classification en mesurant la corrélation réelle entre ses prédictions et la réalité sans risque de biais il se situe entre -1 et 1 : -1 reflétant un modèle faux 0 un modèle qui prédit par hasard et 1 un modèle avec des prédictions fiables.

– État de l'art (étude bibliographique) synthétique présentant les différentes approches et algorithmes ML utilisés pour le traitement de la problématique en question

La littérature existante sur le sujet de la détection de fraude via les algorithmes de machine learning a recensé plusieurs modèles phares utilisés pour la détection de fraude.

Vaishnavi Nath Dornadula et al. mentionne l'utilisation des modèles suivants pour la détection de fraude : Decision Trees, Naive Bayes Classification, Least Squares Regression, Logistic Regression and SVM et Bin Sulaiman et al. ont mentionné les modèles suivants Random forest, ANN, SVM, k-nearest neighbors. ainsi que des approches hybrides avec xGboost. Enfin il y'a aussi le light GBM qui est mentionné - R, L. (2020, 11 février).

Détection de fraude en temps réel avec LightGBM et SocketIO. Pensée Artificielle.

<https://penseeartificielle.fr/detection-de-fraude-temps-reel-python-lightgbm/>

Les modèles que j'ai choisis d'entraîner :

D'après la littérature traitant le sujet de fraude des cartes bancaires cités ci-dessus j'ai choisis d'entraîner les 10 modèles suivants :

- 1) Naives Bayes
- 2) Logistic Regression
- 3) Decision Tree
- 4) KNN
- 5) SVM
- 6) Adaboost
- 7) Random Forest
- 8) Gradient Boosting
- 9) XGBOOST
- 10) Light GBM

2- Exploration data analysis



Avant de commencer le développement et l'entraînement des modèles j'ai exploré les différentes caractéristiques de la base de données Kaggle à ma disposition afin de mieux cerner les besoins en data pre-processing à entamer pour avoir une base d'entraînement fiable.

Profils général du dataset :

```
[4]: df.shape
```

```
[4]: (284807, 31)
```

```
[5]: df.columns
```

```
[5]: Index(['Time', 'V1', 'V2', 'V3', 'V4', 'V5', 'V6', 'V7', 'V8', 'V9', 'V10',
          'V11', 'V12', 'V13', 'V14', 'V15', 'V16', 'V17', 'V18', 'V19', 'V20',
          'V21', 'V22', 'V23', 'V24', 'V25', 'V26', 'V27', 'V28', 'Amount',
          'Class'],
          dtype='object')
```

Le shape montre un dataset assez conséquent avec 284807 lignes et 31 colonnes représentant les variables explicatives. À noter que toutes les variables ont subi une réduction de dimension (PCA) sauf les variables time & amount - selon la description Kaggle.

df.info() : output :

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 284807 entries, 0 to 284806
Data columns (total 31 columns):
# Column Non-Null Count Dtype
-----
0 Time 284807 non-null float64
1 V1 284807 non-null float64
2 V2 284807 non-null float64
3 V3 284807 non-null float64
4 V4 284807 non-null float64
5 V5 284807 non-null float64
6 V6 284807 non-null float64
7 V7 284807 non-null float64
8 V8 284807 non-null float64
9 V9 284807 non-null float64
10 V10 284807 non-null float64
11 V11 284807 non-null float64
12 V12 284807 non-null float64
13 V13 284807 non-null float64
14 V14 284807 non-null float64
15 V15 284807 non-null float64
16 V16 284807 non-null float64
17 V17 284807 non-null float64
```

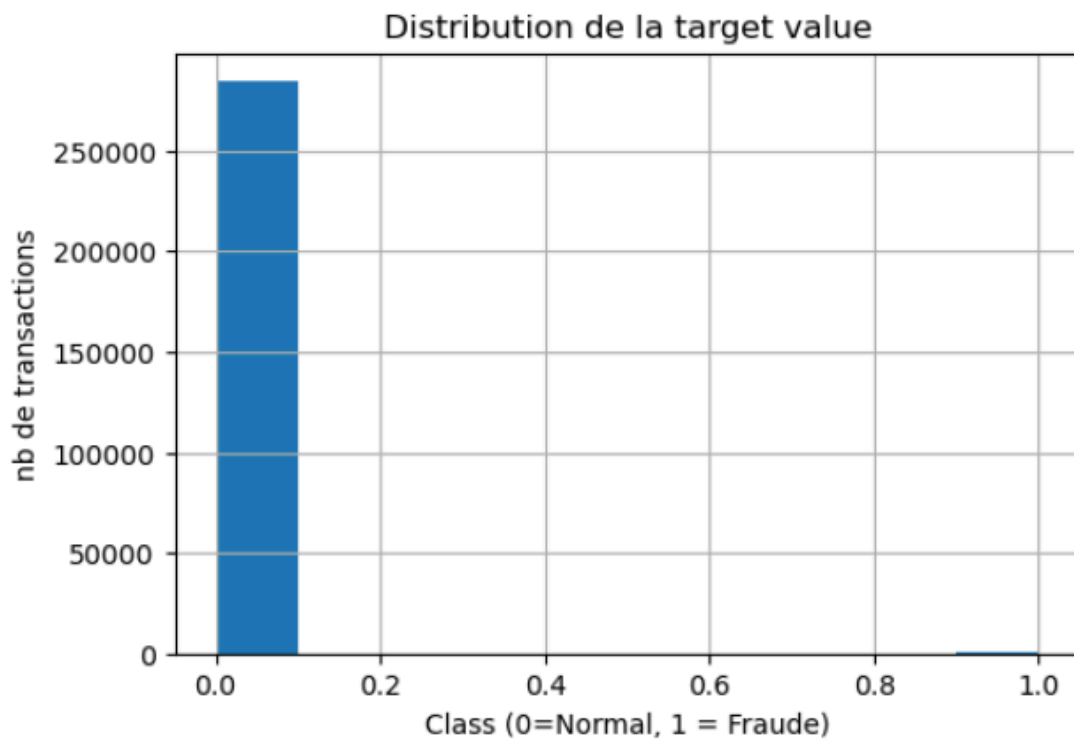
df.isnull().sum() : output

```
Time 0
V1 0
V2 0
V3 0
V4 0
V5 0
V6 0
V7 0
V8 0
V9 0
V10 0
V11 0
V12 0
V13 0
V14 0
V15 0
V16 0
V17 0
V18 0
V19 0
V20 0
V21 0
V22 0
V23 0
V24 0
V25 0
V26 0
V27 0
V28 0
Amount 0
Class 0
dtype: int64
```



```
18 V18 284807 non-null float64
19 V19 284807 non-null float64
20 V20 284807 non-null float64
21 V21 284807 non-null float64
22 V22 284807 non-null float64
23 V23 284807 non-null float64
24 V24 284807 non-null float64
25 V25 284807 non-null float64
26 V26 284807 non-null float64
27 V27 284807 non-null float64
28 V28 284807 non-null float64
29 Amount 284807 non-null float64
30 Class 284807 non-null int64
dtypes: float64(30), int64(1)
memory usage: 67.4 MB
```

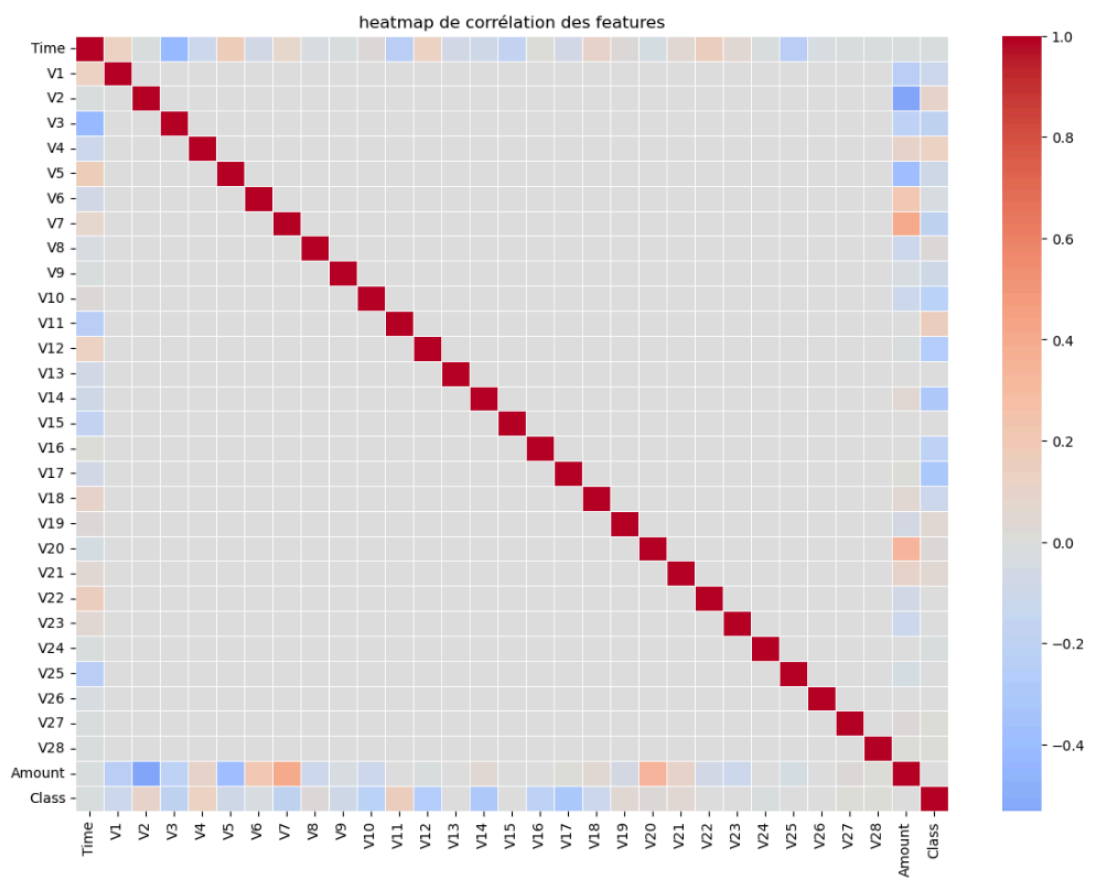
Dans tous le dataset il n'y a aucune valeur nulle donc nous avons pas besoins de traiter la problématique des valeurs nulles dans le dataset





Le dataset contient des données historiques de transaction saines et frauduleuse classifiées sous deux classes : 0 – transaction normale et 1 transaction frauduleuse. D’après le graph ci-dessus on voit bien que le dataset est déséquilibré ce qui ne va pas permettre un bon entraînement des modèles car si on entraîne le modèle sur ce dataset tel quel, il développera un fort biais pour la classe des transactions normale qui est majoritaire. Ceci sera un point à traiter dans le cadre du data – preprocessing.

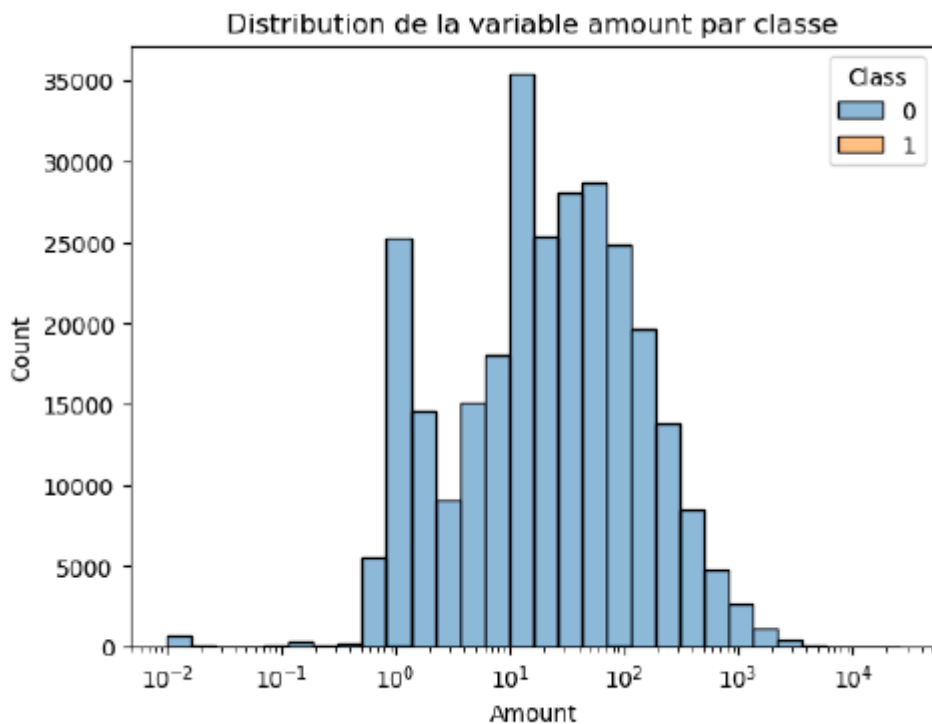
Vérification de la multicollinéarité :



Une des spécificités mentionnées concernant le dataset fourni pour le cas de la détection de la fraude est que toutes les variables ont subi de la PCA sauf time et amount et malgré cela on constate des corrélations modérées de certaines variables avec la variable target ce qui explique la nécessité d’un modèle machine learning pour mieux capter les cas de fraude.



Vu que les variables time et amount sont les seules variables qui n'ont pas subi de transformation, j'ai décidé d'observer leur distribution.



D'après le graphique réalisé ci-dessus on observe que la variable Amount montre une distribution asymétrique avec des écarts importants comme on peut le voir dans le résumé statistique de ce graphique l'entendu entre l'amount min et max est élevé ce qui montre une forte variabilité :



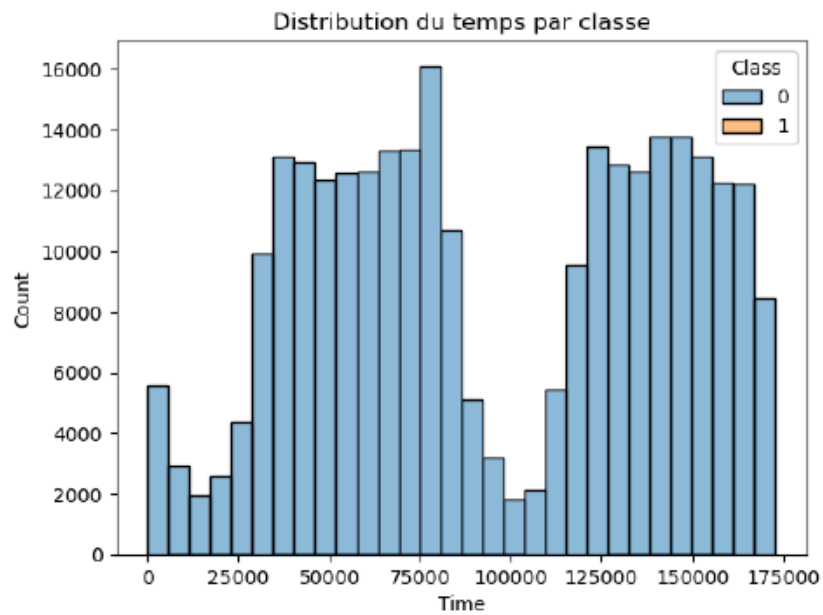
```
[13]:
```

	Amount		mean	std	min	25%	50%	75%	max
Class	count								
0	284315.0		88.291022	250.105092	0.0	5.65	22.00	77.05	25691.16
1	492.0		122.211321	256.683288	0.0	1.00	9.25	105.89	2125.87

	Time		mean	std	min	25%	50%
Class	count						
0	284315.0		94838.202258	47484.015786	0.0	54230.0	84711.0
1	492.0		80746.806911	47835.365138	406.0	41241.5	75568.5

	75%	max
Class		
0	139333.0	172792.0
1	128483.0	170348.0

De ce fait compte tenu de la variabilité constatée, on déduit que il faut une utilisation de Robust Scaler lors de l'étape de pre-processing sinon cette variance et l'existence de valeurs assez de grandeurs différentes risquent de fausser les calculs de distance pour KNN par exemple.

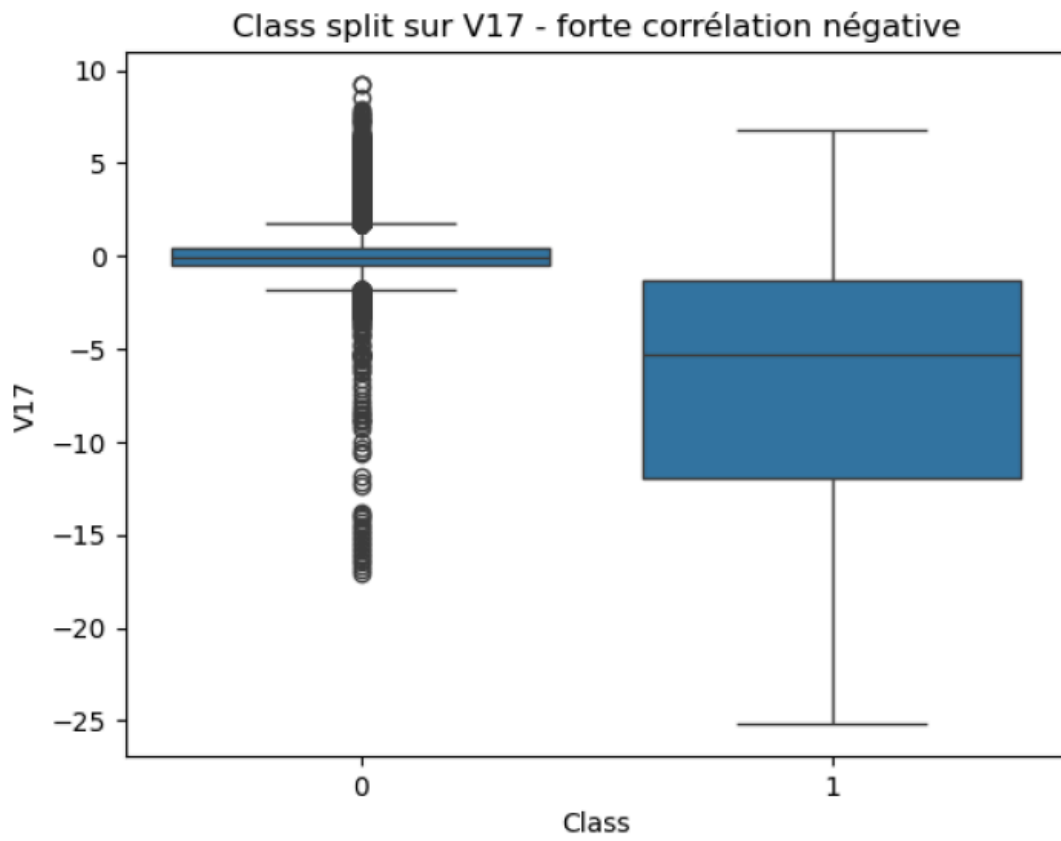


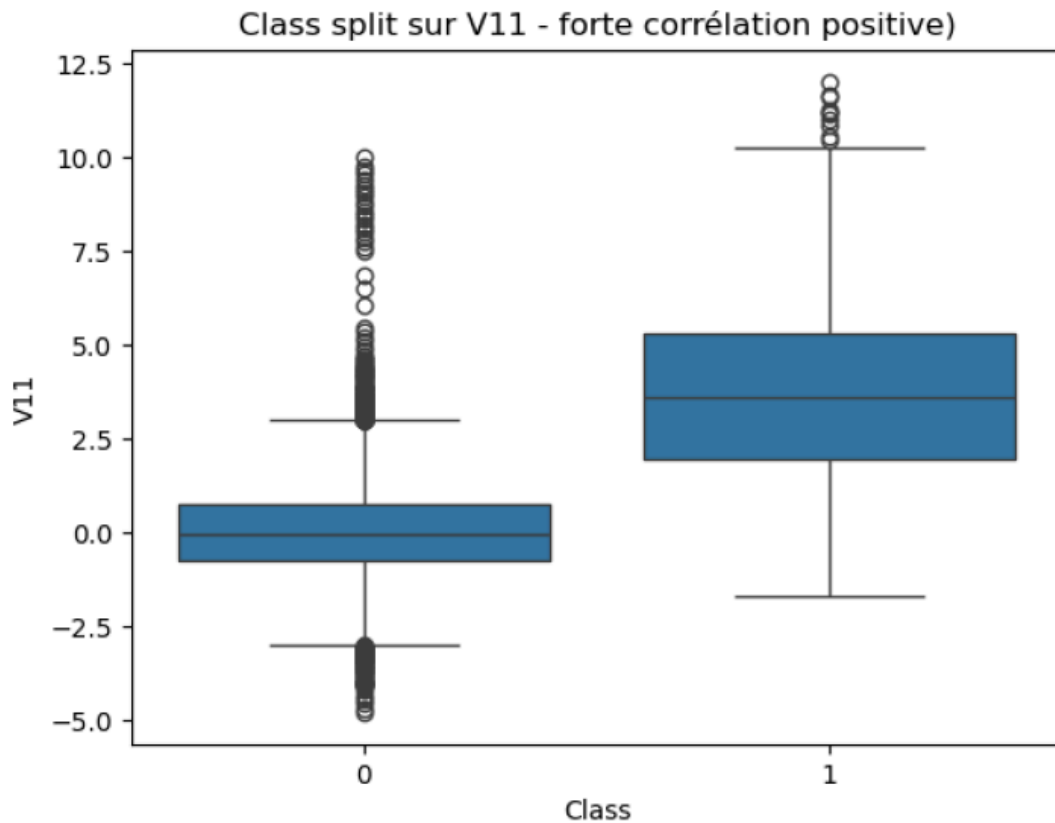
[13]:

	Amount							
	count	mean	std	min	25%	50%	75%	max
Class								
0	284315.0	88.291022	250.105092	0.0	5.65	22.00	77.05	25691.16
1	492.0	122.211321	256.683288	0.0	1.00	9.25	105.89	2125.87

	Time					
	count	mean	std	min	25%	50%
Class						
0	284315.0	94838.202258	47484.015786	0.0	54230.0	84711.0
1	492.0	80746.806911	47835.365138	406.0	41241.5	75568.5

	75%	max
Class		
0	139333.0	172792.0
1	128483.0	170348.0





Dans ces boxplot j'ai effectué un zoom sur les variables qui présentent les coefficients de corrélation les plus élevés (positivement ou négativement) avec la variable target qui 'Class'.

C'est boxplot des variables V14 et V11 montrent que c'est ces deux variables qui vont permettre à l'algorithme de machine learning de bien déterminer si une transaction est frauduleuse ou non puisque les deux médianes des boxplots des deux classes ne se chevauchent pas du tout.

Data pre-processing :

D'après l'analyse exploratoire, plusieurs manipulations de data pre-processing doivent être prises en compte et appliquées avant l'entraînement des modèles. Tout d'abord, le dataset ne contient pas de valeurs manquantes. Cependant, lorsque on a analysé la variable target, on constate un déséquilibre entre les transactions normales/saines et les transactions frauduleuses avec une classe majoritaire contenant des transactions normales. Si ce déséquilibre n'est pas traité, il risque de biaiser les modèles qui vont se focaliser sur la classe 0 et délaisser la classe 1 frauduleuse. Il faut donc rééquilibrer les classes. Pour cela, j'ai procédé à un oversampling



de la classe 1 qui est minoritaire seulement sur les données d'entraînement en utilisant le SMOTE. Ensuite, les variable non transformées par PCA qui sont Time et Amount n'ont pas la même échelle que le reste des variables du dataset, de ce fait pour les scaler et avoir toutes les variable dans la même echelle lors de l'entraînement j'ai appliqué le RobustScaler. Enfin, le pre-processing expliqué et que j'ai prévu pour avoir une bonne dataset d'entraînement a été appliqué après la séparation entre les données test et train pour éviter le data leakage.

De ce fait, j'ai pris en compte ces constats pour entamer mon data processing comme suit :

- 1) J'ai divisé le dataset en deux : dataset d'entraînement contenant 80% des données et dataset de test contenant 20% des données. J'ai utilisé stratify=y pour m'assurer d'avoir une bonne proportion des cas de fraudes dans les deux dataset.
- 2) J'ai ensuite appliqué le RobustScaler pour faire face à la forte variabilité des variables Amount et time, j'ai créé deux variables qui reprennent ce scaling et j'ai supprimé les anciennes variables.
- 3) Enfin, j'ai choisi d'appliquer les SMOTE sur les données d'entraînement afin de générer des données synthétiques pour avoir une parité parfaite afin de garantir un apprentissage non biaisé des 10 modèles.

On peut voir ci-dessous l'effet du SMOTE :

results before SMOTE:

Class

0 227451

1 394

Name: count, dtype: int64

results after SMOTE:

Class

0 227451

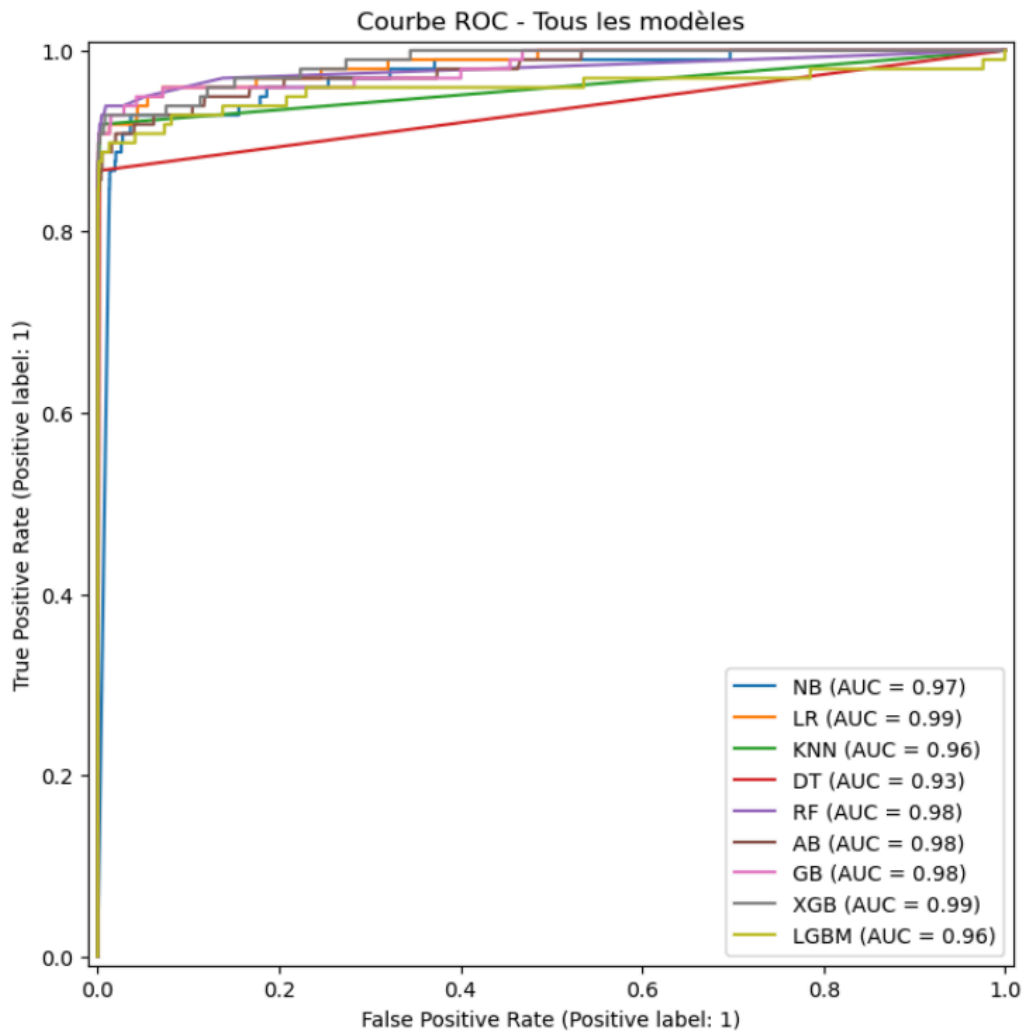
1 227451

Name: count, dtype: int64

Résultat et metrics des 10 modèles après l'entraînement :



]:	Accuracy	Precision	Recall	F1-score	AUC	MCC
NB	0.974	0.056	0.898	0.106	0.936	0.221
LR	0.975	0.059	0.918	0.112	0.947	0.230
KNN	0.998	0.447	0.898	0.597	0.948	0.633
SVM	0.978	0.067	0.918	0.126	0.948	0.246
DT	0.998	0.431	0.867	0.576	0.933	0.611
RF	0.999	0.825	0.867	0.846	0.934	0.846
AB	0.962	0.040	0.908	0.076	0.935	0.185
GB	0.988	0.119	0.908	0.210	0.948	0.326
XGB	0.999	0.711	0.878	0.785	0.938	0.789
LGBM	0.998	0.500	0.878	0.637	0.938	0.662



constat courbe ROC :

On constate que la majorité des courbes sont proches du coin supérieur gauche avec de très bon score AUC. Il a été donc important de confronter cela avec la matrice de confusion pour trancher sur les 3 meilleurs modèles à sélectionner pour le tuning.

Confusion matrix résultats :



*****NB*****

-----Train-----

[[221732 5719]
[35545 191906]]

-----Test-----

[[55384 1480]
[10 88]]

*****LR*****

-----Train-----

[[221846 5605]
[16788 210663]]

-----Test-----

[[55440 1424]
[8 90]]

*****KNN*****

-----Train-----

[[227165 286]
[0 227451]]

-----Test-----

[[56755 109]
[10 88]]

*****SVM*****

-----Train-----

[[222629 4822]
[19694 207757]]

-----Test-----

[[55620 1244]
[8 90]]



*****DT*****

-----Train-----

```
[[227451    0]
 [    0 227451]]
```

-----Test-----

19

```
[[56752   112]
 [   13    85]]
```

*****RF*****

-----Train-----

```
[[227451    0]
 [    0 227451]]
```

-----Test-----

```
[[56846    18]
 [   13    85]]
```

*****AB*****

-----Train-----

```
[[218979  8472]
 [ 14180 213271]]
```

-----Test-----

```
[[54711  2153]
 [    9    89]]
```




*****GB*****

-----Train-----

```
[[224788 2663]
 [ 5515 221936]]
```

-----Test-----

```
[[56203 661]
 [ 9 89]]
```

*****XGB*****

-----Train-----

```
[[227449 2]
 [ 0 227451]]
```

-----Test-----

```
[[56829 35]
 [ 12 86]]
```

*****LGBM*****

-----Train-----

```
[[227213 238]
 [ 0 227451]]
```

-----Test-----

```
[[56778 86]
 [ 12 86]]
```



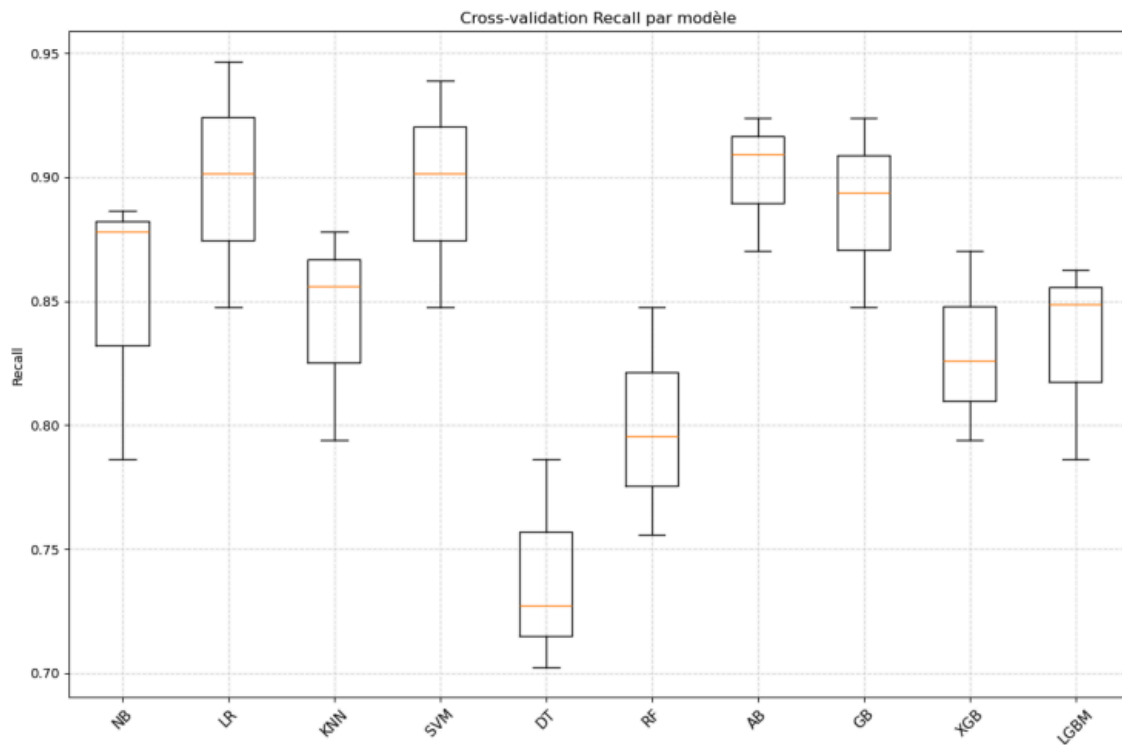
Constat, interprétation des résultats (entraînement, confusion matrix et courbe AUC)

Au vu des résultats des 10 modèles présentés ci-dessus, l'accuracy n'est pas un élément différenciant. Cependant la précision montre des différences entre les modèles une différence encore plus précise avec le F1 score, le MCC et l'AUC. En effet, en se focalisant sur ces métriques, on constate que Naïve Bayes, la régression logistique, SVM et AdaBoost sont pas performant puisqu'ils présentent un F1 score et une précision extrêmement faible avec un MCC faible également donc leur prédiction ne sont principalement pas correctes. Puis on voit des modèles moyens comme KNN, Decision tree, et le gradient boosting qui ont un F1 score moyen entre 0,21 et 0,60 il n'y a pas le bon équilibre entre Recall et précision. Enfin, Random Forest, XGBoost et LightGBM ont tous les trois des F1 score très solides et des Recall très solides également avec de bon AUC.

Cross - validation :

NB : 0.850 ± 0.045
LR : 0.898 ± 0.041
KNN : 0.843 ± 0.036
SVM : 0.896 ± 0.038
DT : 0.739 ± 0.035
RF : 0.800 ± 0.038
AB : 0.901 ± 0.023
GB : 0.888 ± 0.031
XGB : 0.830 ± 0.031
LGBM : 0.832 ± 0.033

cross-validation boxplot :



Constat au niveau de la cross validation :

Les résultats numériques de la cross validation montrent que Adaboost obtient la moyenne de Recall la plus élevée avec l'écart-type le plus faible, il montre de ce fait une grande stabilité et capte bien les fraudes dans la cross validation. Le Decision Tree est le moins performant. Les modèles de Naïf bayes, logistic regression et SVM ont la deuxième moyenne la plus élevée mais présentent un écart-type élevé ce qui fait qu'ils ne sont pas stables. Random forest, XGB et LGBM sont les seuls modèles ayant une stabilité élevée avec une super bonne moyenne. Adaboost, n'a pas été efficace selon la confusion matrix il génère beaucoup de faux positifs et plus de faux négatifs que le trio Random forest, XGB et LGBM qui en génère 0. De plus RF, XGB et LGBM ont les meilleurs F1-score démontrant un bon compromis entre le recall et la précision choses que les autres modèles dont Adaboost n'ont pas montrées.

Au vu de tous les constats relevés de cette phase de modélisation et d'entraînement et d'évaluation des 10 modèles j'ai conclu que les 3 meilleurs modèles qui feront l'objet de tuning sont : Random Forest, Light GBM et XG - boost.

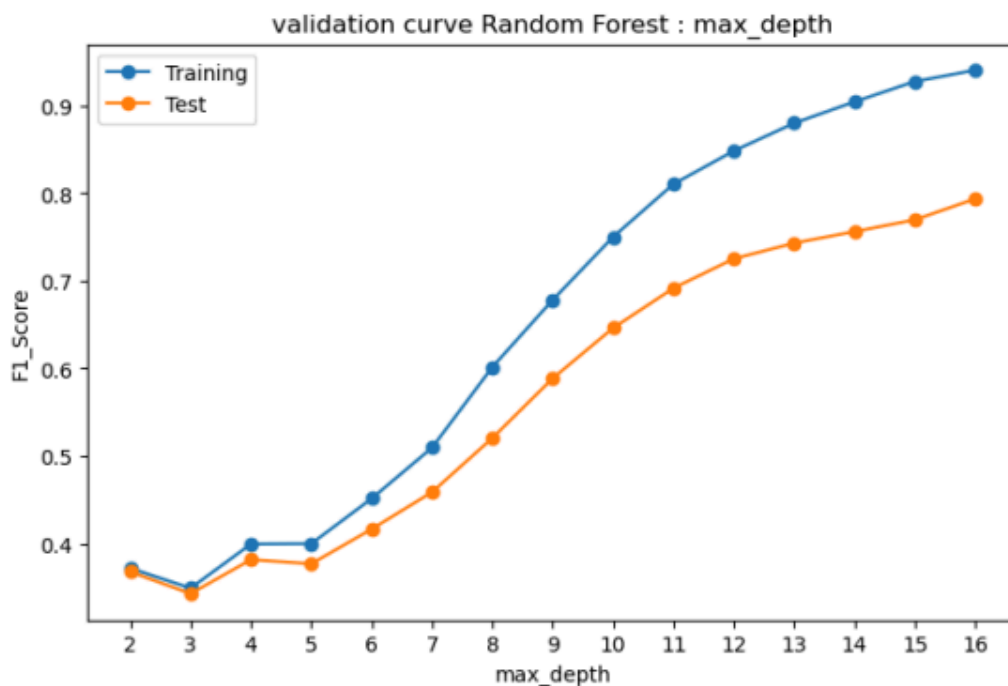


5 Tuning des modèles candidats et choix du modèle final

1) Tuning des modèles candidats :

Comme conclu précédemment en prenant compte de l'ensemble des indicateurs révélés lors de la phase d'entraînement et de test, j'ai shortlisté Random Forest, XGBoost et LightGBM pour leur stabilité et performance. Cependant, afin de maximiser leur performance pour trouver le meilleur modèle présentant le meilleur compromis entre la minimisation des fausses alertes et la détection des fraudes, j'ai procédé à un tuning des hyperparamètres via Randomized Search. j'ai choisi Randomized Search plutôt que Grid Search afin d'optimiser l'identification des paramètres compte tenu des limites de ma machine. j'ai aussi inclus comme critère de recherche $\text{scoring} = 'f1'$ pour avoir une optimisation de performance qui prend en compte l'équilibre entre le Recall et la précision.

Analyse des courbes de validation - tuning part



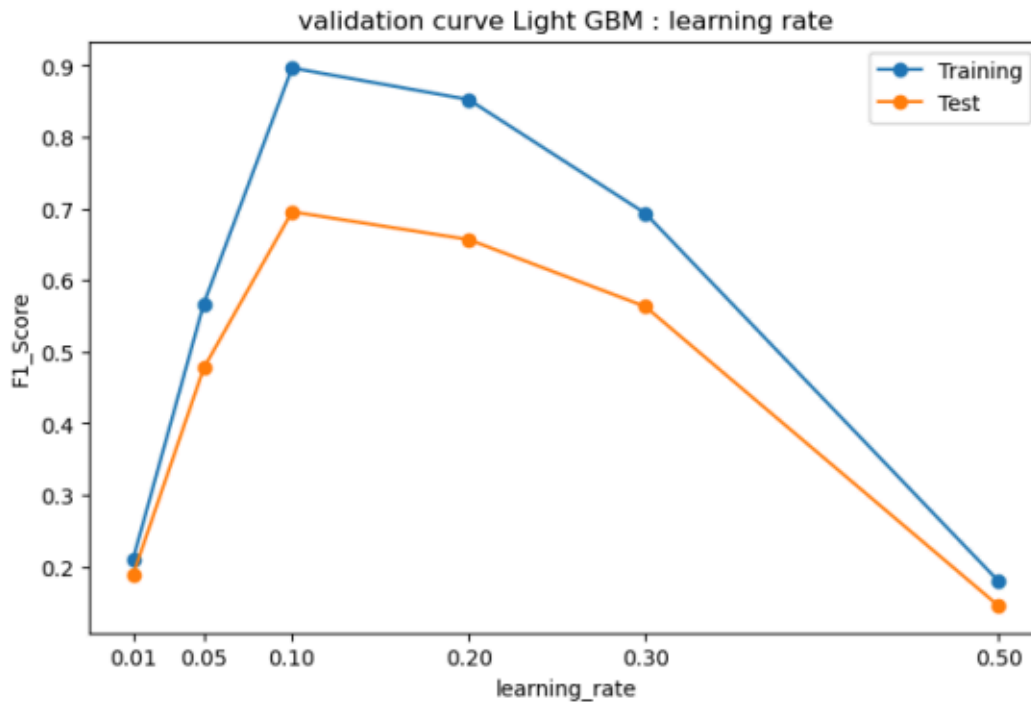
La courbe de validation de Random Forest met en évidence un overfitting apparent dès que la profondeur dépasse 10 niveaux. De ce fait pour le tuning de Random Forest j'ai choisis des petite valeur : 8, 10, 12 lors du Randomized search, 3 folds et 5 fit était le max que pouvais gérer ma machine sans planter donc je pouvais pas jouer sur le nombre d'itérations ni sur le nombre d'estimateurs - lorsque les résultats de cette configuration avec ce nombre d'itération



est sortie j'ai tenté d'augmenter le nombre d'itération après avoir constaté les resultat apres le tuning sur 5 folds mais cela plantais pour Random forest donc je suis resté sur la configuration initiale et j'ai pas essayé d'augmenter pour les autres sinon cela fausserait la comparaison.

La courbe de validation de XGBOOST et LightGBM





Dans la limite des 100 estimateurs le learning rate de XGBOOST et LightGBM à partir duquel les modèle apprennent pour corriger le mieux les erreurs des arbres précédent est de 0,2 et c'est donc en ce sens que j'ai défini les bornes pour la grille de recherche de ces deux modèles.

dans les 3 tuning j'ai utilisé `n_jobs=-1` pour mobiliser toute la force computationnelle de ma machine pour la génération des résultats pour avoir des outputs le plus rapidement possible.

Choix du modèle final avec la mention des résultats finaux du test de ce modèle (metrics finaux, vérifier l'absence d'overfitting, etc.).

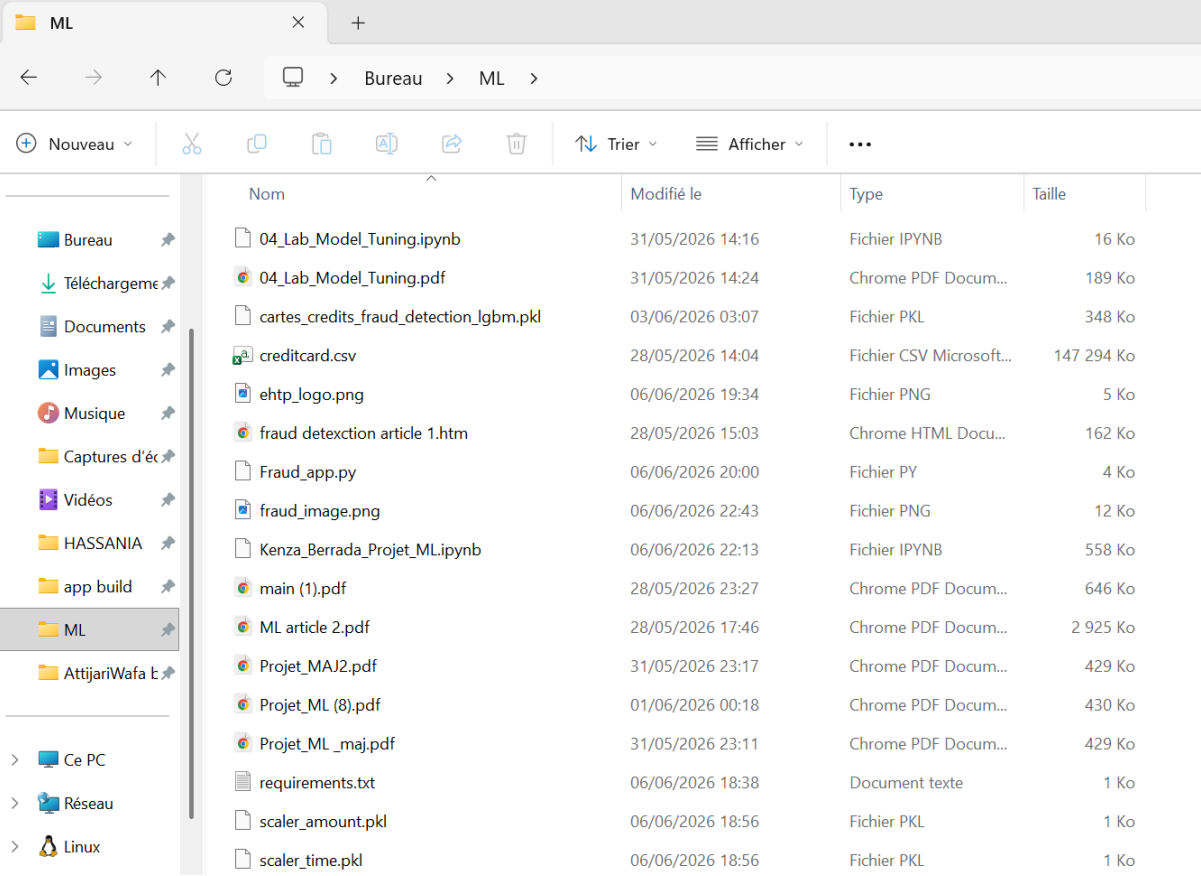
Analyse des résultats après le tuning :

Après le tuning on remarque que le F1 score de Random Forest et XGB baisse alors que celui de LightGBM augmente nettement. Ceci montre que Random Forest et XGB sont sensibles à la variabilité des données qu'ils reçoivent en input ce qui n'est pas le cas pour LightGBM. De ce fait, le modèle que j'ai choisi d'utiliser pour mon application de détection de fraude est le lightGBM.



Déploiement du modèle

Pour mon cas j'ai choisis de sérialiser mon pipeline en deux temps pour être sur de ne pas provoquer des incohérences. dans un premier temps j'ai exporter le modèle intégrant le SMOTE que j'ai utilisé dans mon data pre-processing puis j'ai sérialisé séparément les variable Time and amount qui doivent être calé et je voulais donc m'assurer de bien cibler le scaling sur ces variables uniquement.



The screenshot shows a Windows File Explorer window with the address bar set to 'Bureau > ML'. The left sidebar shows the 'ML' folder selected. The main pane displays a list of files and folders with columns for 'Nom', 'Modifié le', 'Type', and 'Taille'.

Nom	Modifié le	Type	Taille
04_Lab_Model_Tuning.ipynb	31/05/2026 14:16	Fichier IPYNB	16 Ko
04_Lab_Model_Tuning.pdf	31/05/2026 14:24	Chrome PDF Docum...	189 Ko
cartes_credits_fraud_detection_lgbm.pkl	03/06/2026 03:07	Fichier PKL	348 Ko
creditcard.csv	28/05/2026 14:04	Fichier CSV Microsoft...	147 294 Ko
ehtp_logo.png	06/06/2026 19:34	Fichier PNG	5 Ko
fraud detexction article 1.htm	28/05/2026 15:03	Chrome HTML Docu...	162 Ko
Fraud_app.py	06/06/2026 20:00	Fichier PY	4 Ko
fraud_image.png	06/06/2026 22:43	Fichier PNG	12 Ko
Kenza_Berrada_Projet_ML.ipynb	06/06/2026 22:13	Fichier IPYNB	558 Ko
main (1).pdf	28/05/2026 23:27	Chrome PDF Docum...	646 Ko
ML article 2.pdf	28/05/2026 17:46	Chrome PDF Docum...	2 925 Ko
Projet_MAJ2.pdf	31/05/2026 23:17	Chrome PDF Docum...	429 Ko
Projet_ML (8).pdf	01/06/2026 00:18	Chrome PDF Docum...	430 Ko
Projet_ML _maj.pdf	31/05/2026 23:11	Chrome PDF Docum...	429 Ko
requirements.txt	06/06/2026 18:38	Document texte	1 Ko
scaler_amount.pkl	06/06/2026 18:56	Fichier PKL	1 Ko
scaler_time.pkl	06/06/2026 18:56	Fichier PKL	1 Ko

18 élément(s)

Suite à la récupération des différents PKL j'ai rédigé mon [app.py](#) puis j'ajustais l'interface en local et je vérifiais que toutes les fonctionnalités que j'avais prévu dans mon [app.py](#) fonctionnaient correctement.

commande lancée pour visualiser mon app en local :

Berrada Kenza - Projet ML



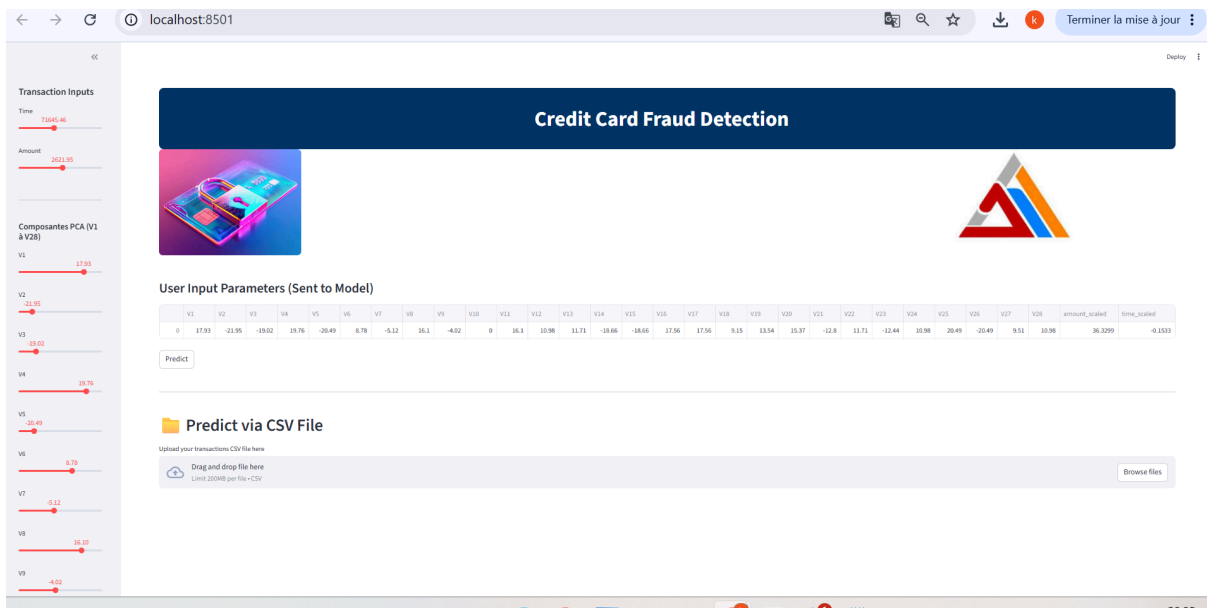
```
Anaconda Prompt

(base) C:\Users\ASUS>cd C:\Users\ASUS\Desktop\ML

(base) C:\Users\ASUS\Desktop\ML>streamlit run Fraud_app.py
```

Mon interface en local :

Accueil :



Les Output des différent test réalisés dans mon app :

Berrada Kenza - Projet ML



Transaction Inputs

Time

7185.46

Amount

2621.95

Composantes PCA (V1 à V28)

V1

17.83

V2

-21.95

V3

-19.62

V4

19.76

V5

-20.49

V6

8.78

V7

-5.12

V8

36.1

V9

-4.02

V10

0

V11

36.1

V12

10.98

V13

11.71

V14

-18.66

V15

-18.66

V16

17.56

V17

17.56

V18

9.15

V19

13.54

V20

15.37

V21

-12.8

V22

11.71

V23

-12.44

V24

10.98

V25

20.49

V26

-20.49

V27

9.15

V28

10.98

amount_scaled

36.3209

time_scaled

-0.1533

Credit Card Fraud Detection



User Input Parameters (Sent to Model)

V1	V2	V3	V4	V5	V6	V7	V8	V9	V10	V11	V12	V13	V14	V15	V16	V17	V18	V19	V20	V21	V22	V23	V24	V25	V26	V27	V28	amount_scaled	time_scaled	
0	17.83	-21.95	-19.62	19.76	-20.49	8.78	-5.12	36.1	-4.02	0	36.1	10.98	11.71	-18.66	-18.66	17.56	17.56	9.15	13.54	15.37	-12.8	11.71	-12.44	10.98	20.49	-20.49	9.15	10.98	36.3209	-0.1533

Predict

Result

ALERT: Fraudulent Transaction Detected! (Confidence: 100.00%)

Prediction Probabilities

Legitimate (0)	Fraudulent (1)
0	1

Predict via CSV File

Upload your transactions CSV file here

Drag and drop file here
Linux 200000 pos file - CSV

Browse files

Transaction Inputs

Time

0.00

Amount

0.46

Composantes PCA (V1 à V28)

V1

-30.00

V2

-30.00

V3

-30.00

V4

-30.00

V5

-30.00

V6

-30.00

V7

-30.00

V8

-30.00

V9

-30.00

V10

-30.00

V11

-30.00

V12

-30.00

V13

-30.00

V14

-30.00

V15

-30.00

V16

-30.00

V17

-30.00

V18

-30.00

V19

-30.00

V20

-30.00

V21

-30.00

V22

-30.00

V23

-30.00

V24

-30.00

V25

-30.00

V26

-30.00

V27

-30.00

V28

-30.00

amount_scaled

0.9706

time_scaled

-0.995

Credit Card Fraud Detection



User Input Parameters (Sent to Model)

V1	V2	V3	V4	V5	V6	V7	V8	V9	V10	V11	V12	V13	V14	V15	V16	V17	V18	V19	V20	V21	V22	V23	V24	V25	V26	V27	V28	amount_scaled	time_scaled	
0	-30	-30	-30	-28.54	0.37	-6.78	0	0	-4.1	0	-4.1	0	0	0	-1.46	0.37	0	0	-0.95	-12.8	0	0	0	0	0	0	-1.46	-1.1	0.9706	-0.995

Predict

Result

Transaction Approved (Legitimate) (Confidence: 100.00%)

Prediction Probabilities

Legitimate (0)	Fraudulent (1)
1	0.000000

Predict via CSV File

Upload your transactions CSV file here

upload de csv possible :

Berrada Kenza - Projet ML



The screenshot shows a web application running on localhost:8501. The interface includes a sidebar with 'Transaction Inputs' and 'Composantes PCA (V1 à V28)'. The main area displays 'User Input Parameters (Sent to Model)' with a table of 11 features (V1 to V11) and their values. Below this is a 'Predict' button and a 'Result' section showing 'Transaction Approved (Legitimate) (Confidence: 100.00%)'. There is also a 'Prediction Probabilities' section and a 'Predict via CSV File' section with an upload button. A file explorer window is open, showing the 'Bureau' directory with files like '.ipynb_checkpoints' and 'creditcard.csv'.

Github - création de repository

The screenshot shows the GitHub 'Create a new repository' page. The 'General' section is active, showing the 'Owner' as 'kb7ste-lgtn' and the 'Repository name' as 'credit-cards-fraud-detection-ml'. A message indicates that the name is available. The 'Description' field is empty. The 'Configuration' section shows the 'Choose visibility' dropdown set to 'Public'.

Berrada Kenza - Projet ML



github.com/new

credit-cards-fraud-detection-ml is available.

Great repository names are short and memorable. How about [literate-fishstick?](#)

Description

0 / 350 characters

2 Configuration

Choose visibility *

Choose who can see and commit to this repository

Public

Add README

READMEs can be used as longer descriptions. [About READMEs](#)

On

Add .gitignore

.gitignore tells git which files not to track. [About ignoring files](#)

No .gitignore

Add license

Licenses explain how others can use your code. [About licenses](#)

No license

Create repository

Github - création du README :

github.com/kb7ste-lgtm/credit-cards-fraud-detection-ml/edit/main/README.md

kb7ste-lgtm / credit-cards-fraud-detection-ml

Type / to search

Code Issues Pull requests Actions Projects Wiki Security and quality Insights Settings

Files

main

Go to file

README.md

credit-cards-fraud-detection-ml / README.md in main

Cancel changes Commit changes...

Edit Preview

Spaces 2 Soft wrap

```
1 # credit-cards-fraud-detection-ml
2 Il s'agit d'une application streamlit dédiée à la détection de fraudes par carte bancaire.
3 ## Description
4 Ce projet repose sur un modèle LightGBM pour identifier les transactions frauduleuse sur des variables ayant subi un PCA au préalable
sauf deux qui ont été scalé. L'interface utilisateur permet à l'utilisateur d'insérer son input de deux manières : via des barres de
scaling et le traitement /upload de fichier CSV.
5 ## Overview des fonctionnalités :
6 -** interface interactive via les sliders pour tester différentes variables PCA(V1 à V28).
7 -**upload de fichier csv des transactions : ** possibilité d'uploader un fichier csv pour analyser des transactions.
8 -** Design simple et friendly.
9 ## librairies utilisées : Streamlit, Pandas, numpy, Scikit-learn (RobustScaler), LightGBM, Imbalanced-learn
10 ##Auteur
11 Projet réalisé dans le cadre de mon master à l'EHTP.
```

Berrada Kenza - Projet ML



The screenshot shows the GitHub repository page for 'credit-cards-fraud-detection-ml'. The browser address bar displays 'github.com/kb7ste-lgtm/credit-cards-fraud-detection-ml/blob/main/README.md'. The left sidebar shows the 'Files' section with 'main' selected and a search bar. The main content area has tabs for 'Preview', 'Code', and 'Blame'. The 'Preview' tab is active, showing the README content. The README title is 'credit-cards-fraud-detection-ml'. The text describes the application as a Streamlit app for detecting bank card fraud. It includes a 'Description' section stating the project uses a LightGBM model on PCA-transformed data. An 'Overview des fonctionnalités' section lists features like an interactive slider interface and CSV upload capabilities. A 'librairies utilisées' section lists the libraries: Streamlit, Pandas, numpy, Scikit-learn (RobustScaler), LightGBM, and Imbalanced-learn.

credit-cards-fraud-detection-ml

Il s'agit d'une application streamlit dédiée à la détection de fraudes par carte bancaire.

Description

Ce projet repose sur un modèle LightGBM pour identifier les transactions frauduleuse sur des variables ayant subi un PCA au préalable sauf deux qui ont été scalé. L'interface utilisateur permet à l'utilisateur d'insérer son input de deux manières : via des barres de scaling et le traitement /upload de fichier CSV.

Overview des fonctionnalités :

- ** interface interactive via les sliders pour tester différentes variables PCA(V1 à V28).
- upload de fichier csv des transactions :
- ** possibilité d'uploader un fichier csv pour analyser des transactions.

librairies utilisées : Streamlit, Pandas, numpy, Scikit-learn (RobustScaler), LightGBM, Imbalanced-learn

Gutub -Upload des files :

The screenshot shows the GitHub repository upload page for 'credit-cards-fraud-detection-ml'. The browser address bar displays 'github.com/kb7ste-lgtm/credit-cards-fraud-detection-ml/upload/main'. The main content area has a large box with a file icon and the text 'Drag additional files here to add them to your repository' and 'Or choose your files'. Below this, there is a list of files that have been uploaded, each with a file icon and a close button (X):

- Fraud_app.py
- cartes_credits_fraud_detection_lgbm.pkl
- requirements.txt
- fraud_image.PNG
- ehpt_logo.PNG

The Windows taskbar is visible at the bottom of the screen, showing the time as 21:46 on 06/06/2026.

Déploiement sur streamlight cloud :

Berrada Kenza - Projet ML



share.streamlit.io

kb7ste-lgtm ▾ My apps My profile Explore Discuss ↗ Create app

kb7ste-lgtm's apps

model-irisi · main · app.py

Get started from a template

[View all templates →](#)

GDP dashboard View demo

Chatbot View demo

Support tickets View demo

Blank app View demo

kb7ste-lgtm ▾ My apps My profile Explore Discuss ↗

[← Back](#)

Deploy an app

Repository ⓘ

[Paste GitHub URL](#)

kb7ste-lgtm/repo

Branch

master

Main file path

streamlit_app.py

App URL (optional)

.streamlit.app

Berrada Kenza - Projet ML



share.streamlit.io/deploy

kb7ste-lgtm ▾ My apps My profile Explore Discuss ↗

Repository ⓘ [Paste GitHub URL](#)

kb7ste-lgtm/credit-cards-fraud-detection-ml

Branch

main

Main file path

Fraud_app.py

App URL (optional)

credit-cards-fraud-detection-ml-qzseuzpsk48k6ndecibkjg .streamlit.app

Domain is available

[Advanced settings](#)

22:28 06/06/2026

credit-cards-fraud-detection-ml-qzseuzpsk48k6ndecibkjg.streamlit.app

Installer ⓘ ☆ ⬇ 🔴 ⋮



Your app is in the oven

Manage app

22:30 06/06/2026



[Liens](#)

Lien de l'app :

<https://credit-cards-fraud-detection-ml-qzseuzpsk48k6ndecibkjg.streamlit.app/>

lien repository :

<https://github.com/kb7ste-lgtm/credit-cards-fraud-detection-ml>